BV TECH
Securing the future

Anonymization Pipeline

Documentazione tecnica: flusso logico, architettura e componenti

VERSIONE 0.1.0 · OTTOBRE 2026 · PIATTAFORMA MIA · AGENAS

1 SCOPO E PRINCIPI

Anonymization Pipeline è il servizio della piattaforma MIA che anonimizza documenti italiani, soprattutto sanitari: PDF nativi e scansionati, immagini e testo. Restituisce il Markdown del documento censurato, un report senza valori e, in modalità pseudonimo, una mappa cifrata che consente di ripristinare i valori originali solo a chi possiede la chiave.

Il servizio è stateless (nessun database, lavori asincroni in memoria) e deroga al template AGENAS in un solo punto sostanziale: non usa llm-gateway, perché il VLM è locale e i documenti non devono uscire dal perimetro (`docs/DEPLOY_HANDOVER.md`).

5	93	4	0
fasi della pipeline	categorie di dati personali in 13 gruppi	rilevatori in unione	chiamate verso l'esterno a runtime

Principi di progetto

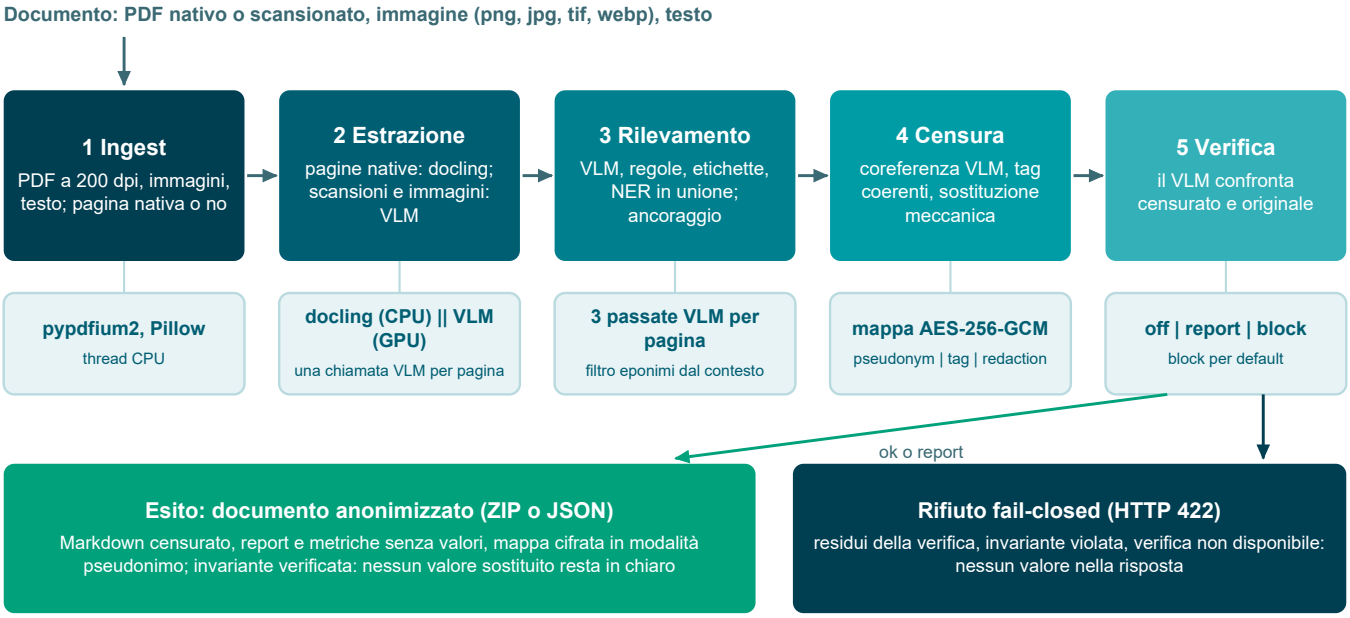
Principio	Come è realizzato
Tutto nel perimetro	VLM locale (Ollama o server compatibile OpenAI), docling e rilevatori su CPU. A runtime nessun accesso a internet: i modelli si scaricano prima, con `just models`.
Fail-closed	Se un dato può restare in chiaro (residui, invariante violata, verifica non disponibile) il documento è rifiutato con HTTP 422, senza valori nella risposta.
Modello non fidato	Ogni valore proposto deve comparire nel testo (ancoraggio), gli altri sono allucinazioni e si scartano. Un'istruzione nascosta nel documento può al più nascondere un residuo, mai far comparire un valore.
Rilevatori in unione	VLM, regole con checksum, etichette e NER lavorano in parallelo: basta un rilevatore per censurare un valore.
Sostituzione meccanica	Il modello decide cosa è sensibile e quali varianti sono la stessa entità; il codice sostituisce ogni forma in tutto il documento con tag coerenti (`[PERSONA_1]`), quindi il ripristino è esatto.
Nessun dato nei log	Report, metriche ed eventi contengono solo conteggi, categorie, pagine e tempi; i codici fiscali sono mascherati anche nei traceback.
Configurazione dichiarativa	Pipeline, categorie, prompt, modelli e vocabolari dei rilevatori sono file YAML validati all'avvio: una chiave sconosciuta ferma il servizio.

Stack tecnologico

Livello	Tecnologie
Backend	Python 3.12, FastAPI e uvicorn, Pydantic e pydantic-settings, httpx, pypdfium2 e Pillow, cryptography (AES-256-GCM, scrypt); uv, ruff, ty, pytest
Modelli	VLM su Ollama o server compatibile OpenAI (GPU); docling e NER Hugging Face con transformers e torch su CPU (extra opzionali)
Interfaccia web	React, TypeScript e Vite, servita da nginx con CSP restrittiva; pdf.js, IndexedDB

2 FLUSSO LOGICO

Ogni documento attraversa cinque fasi in sequenza (`pipeline.py`). Il VLM decide cosa è scritto nelle pagine non native, cosa è sensibile, quali valori sono la stessa entità e cosa è sfuggito; il codice fa I/O, valida l'output dei rilevatori e sostituisce le stringhe. Ogni fase usa il proprio modello, scelto nel file dei modelli (estrazione: glm-ocr; rilevamento, censura, verifica: granite4.2:8b).



Fase	Cosa fa	Garanzie
Ingest `ingest/`	Riconosce il formato dai magic byte, renderizza le pagine PDF (200 dpi) e decide se una pagina è nativa: testo leggibile e visibile, poche immagini, niente testo nascosto. Il testo semplice è diviso in blocchi.	Limiti su pagine e pixel (decompression bomb); file cifrati o corrotti: HTTP 400 con messaggio generico.
Estrazione `extraction/`	Pagine native con docling (testo esatto, tabelle), in parallelo con il VLM che trascrive scansioni e immagini in Markdown fedele, una chiamata per pagina.	Senza docling ogni pagina va al VLM, con un avviso; trascrizione troncata: avviso nel report.
Rilevamento `detection/` `detectors/`	VLM (3 passate di categorie per pagina, risposta vincolata da JSON schema), regole con checksum (CF, IBAN, P.IVA, carte, ICD-10), etichette ("Paziente: ...") e NER, in unione.	Ancoraggio letterale o approssimato; categoria spenta: ripiego sulla generica; eponimi e scale scartati solo nel loro contesto medico.
Censura `redaction/`	Il VLM raggruppa le varianti della stessa entità (coreferenza); il codice sostituisce ogni forma in tutto il documento. Modalità pseudonym (tag e mappa), tag o redaction (simbolo fisso).	Varianti fuse solo nella stessa famiglia; mappa AES-256-GCM, chiave casuale o passphrase con scrypt, intestazione autenticata.
Verifica `verification/`	Il VLM rilegge ogni pagina censurata confrontandola con il Markdown originale e segnala i dati rimasti in chiaro. Poi il controllo dell'invariante sul testo finale.	Modalità off, report o block (default); verifica obbligatoria: se il VLM non risponde il documento è rifiutato.

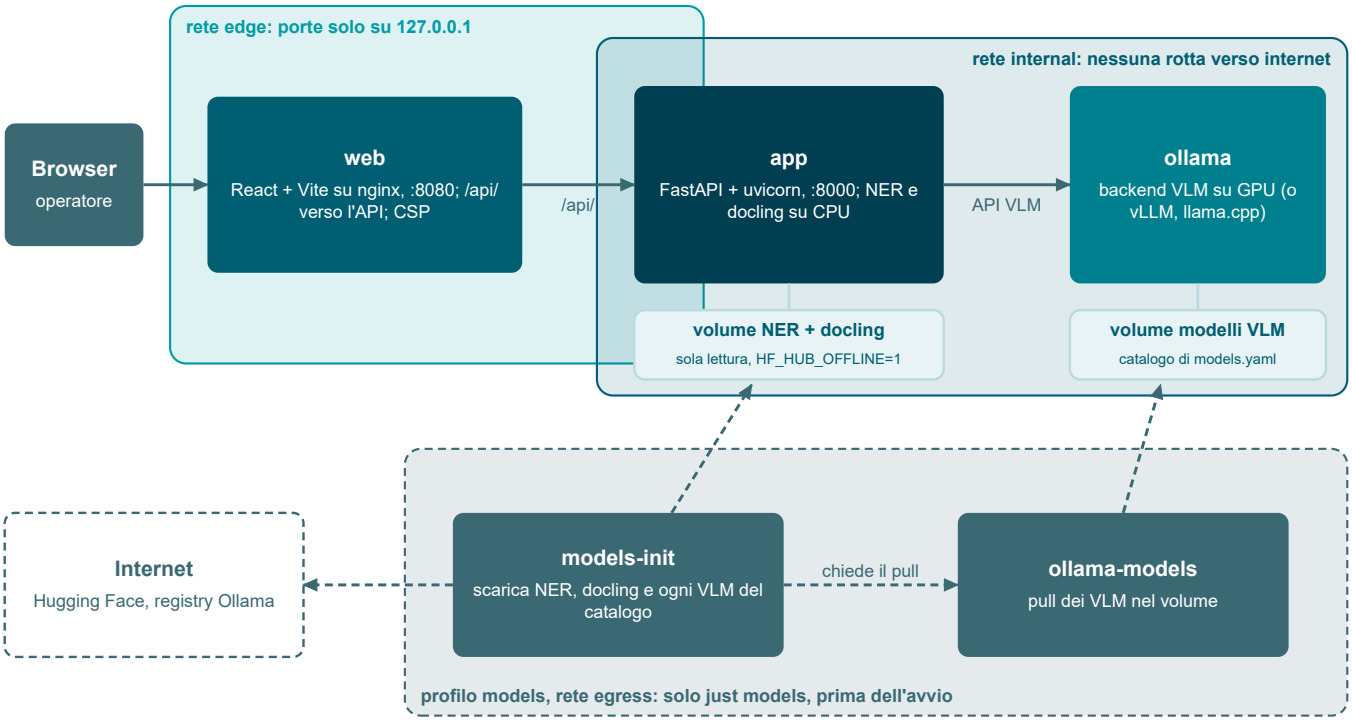
Esiti e ripristino

`POST /anonymize` restituisce uno ZIP (default) o JSON con `<nome>.anon.md`, `.report.json`, `.metriche.json` e, in modalità pseudonimo, `.mappa.enc`. `POST /restore` ricostruisce il documento originale da Markdown censurato, mappa e chiave (`/restore/detail` con le posizioni dei tag). Chiave errata o mappa alterata: HTTP 403.

Confidenziale. Questo documento contiene informazioni riservate e non può essere riprodotto o diffuso senza autorizzazione.

3 VISTA ARCHITETTURALE

Tre container in esecuzione su due reti; un profilo separato, l'unico con accesso a internet, scarica i modelli prima dell'avvio. Nel cluster le stesse regole sono NetworkPolicy (`docs/k8s-example/service.yaml`).



Isolamento e deploy

Compose (sviluppo)	Kubernetes (staging e produzione)
`ollama` solo sulla rete internal, senza porte pubblicate; `app` raggiunge solo URL interni, legge i modelli con `HF_HUB_OFFLINE=1` e non chiede mai download a Ollama (`ANONPIP_VLM_PULL_MISSING=false`); porte su 127.0.0.1.	Immagini API e web su Harbor, Flux CD. Esposizione interna (traefik-int / WSO2), probe `/healthz` e `/readyz` (backend VLM e modello di ogni fase), 1 replica, backend VLM con 1 GPU NVIDIA, PVC dei modelli in sola lettura, nessun database.

Vista runtime

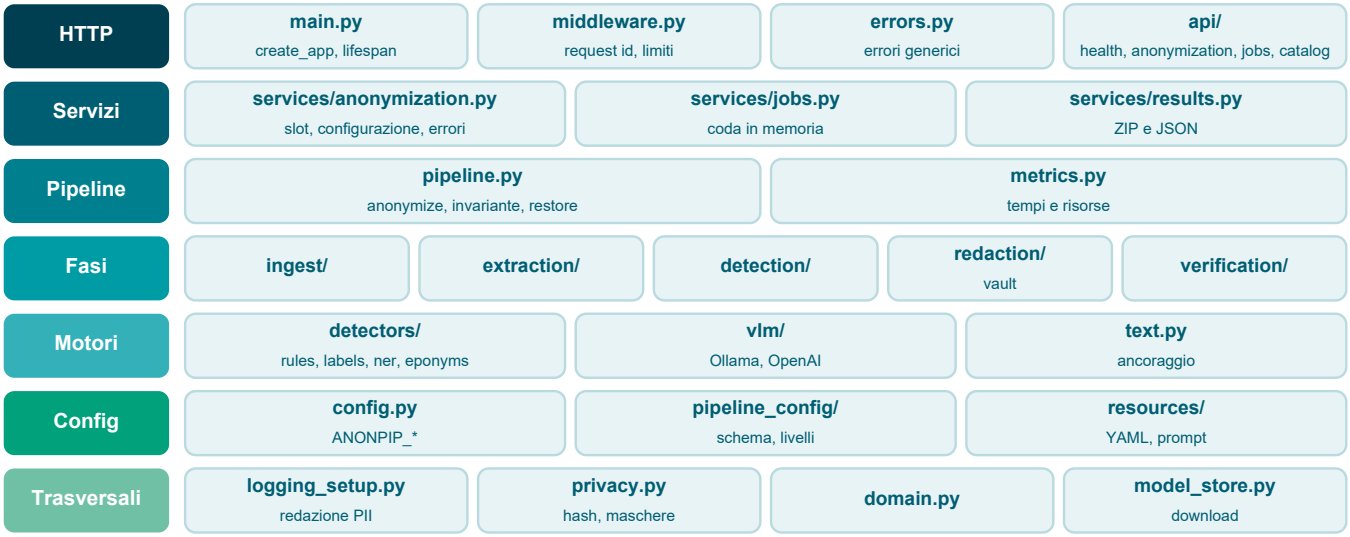
La GPU è il collo di bottiglia: i documenti elaborati insieme sono limitati da `ANONPIP_MAX_CONCURRENT_JOBS` (1). Una sola replica e un solo worker uvicorn, perché i lavori asincroni vivono nella memoria del processo; i byte del file e la chiave sono rilasciati a fine elaborazione.

	Richiesta sincrona	Lavoro asincrono
Endpoint	`POST /anonymize`	`POST /jobs`, poi `GET /jobs/{id}` e `GET /jobs/{id}/result`
Concorrenza	attende uno slot fino a `ANONPIP_JOB_WAIT_S` (30 s), poi HTTP 503 con Retry-After	coda in memoria (max `ANONPIP_MAX_QUEUED_JOBS`, 20), stessi slot delle richieste sincrone
Stato	nessuno: la risposta contiene il risultato	fase corrente e chiamate VLM; risultato fino alla scadenza (1 ora) o alla cancellazione
Uso	integrazione fra servizi, script	interfaccia web

Confidenziale. Questo documento contiene informazioni riservate e non può essere riprodotto o diffuso senza autorizzazione.

4 COMPONENTI

Il backend segue il layout del template: `api/` (router) → `services/` (logica) → `pipeline.py` → moduli di fase, che usano i motori (rilevatori, client VLM, ancoraggio) e la configurazione. Le dipendenze scendono soltanto; i rilevatori non si conoscono fra loro.



Componente	Responsabilità
`api/`	Router sottili: validano i campi (modelli Pydantic con limiti) e delegano ai servizi. `/anonymize`, `/restore`, `/jobs`, `/models`, `/categories`, `/configuration`, probe.
`services/anonymization.py`	Configurazione della richiesta, slot di concorrenza, mappatura degli errori su HTTP, stato per /readyz; all'avvio valida configurazione e file dei rilevatori.
`services/jobs.py`	Coda in memoria, worker, avanzamento per fase, scadenza dei risultati; eventi job_queued e job_finished.
`pipeline.py`	Orchestrazione delle fasi, controllo dell'invariante, unione delle pagine, report, cifratura della mappa, ripristino.
`detectors/`	Rilevatori di testo indipendenti: rules (regex e checksum), labels (etichetta e valore), ner (modello Hugging Face su CPU), eponyms (filtro contestuale); `patterns.py`: sintassi comune delle regex.
`vlm/`	Client per Ollama (API nativa, num_ctx) e per server compatibili OpenAI; codifica delle immagini, retry, parsing tollerante delle risposte JSON.
`text.py`	Ancoraggio dei valori al Markdown (letterale, case-insensitive, approssimato) e riconoscimento dei tag già inseriti.
`pipeline_config/`	Schema Pydantic della pipeline, caricamento a livelli, registro delle categorie, catalogo dei modelli.
`logging_setup.py`, `privacy.py`	Formato dei log per Loki, filtro che maschera i codici fiscali, hash e maschere degli identificativi, eventi KPI.
`model_store.py`	Download di VLM, NER e docling prima dell'avvio: l'unico passo con internet.

Pagina web	Funzione
Carica / Stato	Invio del documento con modalità, modelli per fase, rilevatori e categorie; stato del servizio e dei modelli.
Confronto	Originale, Markdown in chiaro e censurato affiancati, con i tag evidenziati.
Dashboard / Storico	Tempi, chiamate VLM e conteggi; storico in IndexedDB con soli report e metriche, mai il nome del file.
Ripristino	Markdown e mappa con la chiave, che resta solo in sessionStorage.

Confidenziale. Questo documento contiene informazioni riservate e non può essere riprodotto o diffuso senza autorizzazione.

5 CONFIGURAZIONE, SICUREZZA, OPERATIVITÀ

La configurazione effettiva è la fusione di cinque livelli: il file utente è indicato da `ANONPIP_PIPELINE_CONFIG_FILE`, il file dei modelli da `ANONPIP_MODELS_FILE`. Ogni chiave sconosciuta è un errore e le chiavi possedute dalle variabili o dal file dei modelli sono rifiutate nel file utente. `GET /configuration` mostra la configurazione effettiva senza segreti.



più debole — più forte: vince sulle precedenti

File in resources/	Contenuto
`default.yaml`	Parametri regolabili: backend, generazione, immagini, fasi, rilevatori accesi, soglie, output.
`models.yaml`	Catalogo dei VLM (contesto, visione) e modello predefinito di ogni fase; sostituibile con un file proprio.
`categories.yaml`	Registro delle 93 categorie, gruppi, famiglie generiche, coreferenza libera.
`prompts/<fase>.yaml`	Prompt di sistema e utente, con i segnaposto validati.
`detectors/*.yaml`	Vocabolari dei rilevatori (regole, etichette, NER, eponimi): validati all'avvio, non sovrascrivibili dal file utente.

Privacy e sicurezza	Osservabilità e operatività
Log in chiaro solo con conteggi e hash; testo dell'utente solo in DEBUG fuori produzione e redatto; codici fiscali mascherati da un filtro su tutto lo stdout. Nessun dato personale in URL. Errori generici in italiano, mai stack trace o input. Limite di upload (413/411), X-Request-ID validato, OpenAPI e trascrizione in chiaro disattivate in produzione. nginx: CSP, X-Frame-Options DENY, no-referrer.	Eventi document_anonymized, document_rejected, job_queued e job_finished verso Loki (contratto con KPI-dashboard). Metriche per documento: tempi per fase e pagina, latenze e token del VLM, CPU e memoria. `/readyz` riporta backend VLM, modelli, NER e docling.

Risposta	Quando (messaggio generico in italiano, mai valori del documento)
400	Documento non valido, cifrato o in formato non supportato; parametro non valido (modello fuori catalogo, categoria sconosciuta).
403 / **404** / **409**	Ripristino non riuscito (chiave o mappa); lavoro inesistente o scaduto; risultato richiesto prima della fine del lavoro.
411 / **413**	Upload senza lunghezza o oltre ANONPIP_MAX_UPLOAD_MB.
422	Rifiuto fail-closed: residui, invariante violata, verifica non disponibile; nel corpo solo categorie e pagine.
503	Backend VLM o modello non disponibile; servizio occupato (con Retry-After).

Comando	Effetto
`just models`	Scarica VLM, NER e docling (unico passo con internet).
`just up-gpu` / `just up`	Avvia lo stack compose su GPU o su CPU.
`just lint` / `just test-cov`	ruff e ty; unit test con coverage minima 80%.
`just test-vlm`	Test di integrazione contro l'Ollama del compose.
`just docs-pdf`	Rigenera questo documento.

Confidenziale. Questo documento contiene informazioni riservate e non può essere riprodotto o diffuso senza autorizzazione.